USEI13 — Rail Hub Centrality Analysis

(CentralityCalculator)

Algorithm: `CentralityCalculator.computeHubRanking()`

Time Complexity: $O(V \times (E + V \log V))$ or $O(V^3)$ in worst case (dense graph)
Space Complexity: $O(V^2)$ in worst case

Where:

- V = Number of vertices (Stations) in the graph
- E = Number of edges (TrackConnection) in the graph

Detailed Analysis:

1. Degree/Strength Calculation (`centrality()`)

- Iterate over all vertices → $O(V)$
- For each vertex, compute `outDegree + inDegree` → $O(1)$ per vertex (assuming adjacency list)
- **Total:** $O(V)$

2. Betweenness Centrality (`betweennessCentrality()`)

- For each source vertex (V times):
    - Run **shortest paths from source** (Dijkstra or BFS variant) → $O(E + V \log V)$ with priority queue
    - For each destination (V times):
        - Process path to increment betweenness → $O(\text{path length}) \leq O(V)$
- **Betweenness per source-destination pair:** $O(V)$ path processing in worst case

- **Total worst case:** $O(V \times (E + V \log V) + V^2 \times V) = O(V \times (E + V \log V) + V^3)$
  Simplifies to $O(V^3)$ for dense graphs where $E \approx V^2$.

3. Harmonic Closeness Centrality (`closenessCentrality()`)

- For each source vertex (V times):
  - Run **shortest paths from source** → $O(E + V \log V)$
  - Sum inverse distances for all destinations → $O(V)$
- **Total:** $O(V \times (E + V \log V))$

4. Normalization Steps (`normalize()` and `normalizeInt()`)

- Find min/max: $O(V)$
- Apply normalization: $O(V)$
- Called 3 times → **Total:** $O(V)$

5. Hub Score Calculation & Sorting

- Compute weighted sum for each vertex → $O(V)$
- Sort results → $O(V \log V)$
- **Total:** $O(V \log V)$

Overall Time Complexity:

Dominant terms: Betweenness ($O(V^3)$) + Closeness ($O(V \times (E + V \log V))$)

For sparse graphs ($E \approx V$):
$O(V \times (V + V \log V)) + O(V \times (V + V \log V)) = O(V^2 \log V)$

For dense graphs ($E \approx V^2$):
$O(V^3) + O(V^3) = O(V^3)$

Final: $O(V^3)$ in worst case (dense networks)

Space Complexity: $O(V^2)$ in worst case

Breakdown:

- **Graph storage:** $O(V + E)$
- **Betweenness computation:**
  - Shortest paths may store $O(V^2)$ path information in worst case
  - Distance arrays: $O(V)$
- **Maps for metrics:** $O(V)$ each (degree, betweenness, closeness) → $O(V)$ total
- **Sorted results list:** $O(V)$

Dominant Factor: Shortest path storage for betweenness calculation → $O(V^2)$ in worst case.

Summary for USE113:

- **Time Complexity:** $O(V^3)$ worst case – cubic scaling with station count
- **Space Complexity:** $O(V^2)$ – quadratic storage for path information
- **Practical Implication:** Suitable for moderate-sized networks (≤ 500 stations), may require optimization for national-scale networks.